1. Overview

1.1 Project Objectives and System Architecture

The primary objective of this project is to develop a comprehensive, high-interaction web application that addresses the "last-mile" transportation challenges in Dublin by integrating real-time data, meteorological information, and predictive analytics. The system provides users with live occupancy status for Dublin Bikes stations while leveraging machine learning to assist in travel decision-making under Dublin's unpredictable weather patterns.

Architecturally, the project utilizes a decoupled client-server and cloud architecture, strictly separating front-end user interfaces, back-end web routing, and data acquisition processes. The work is split across three independent Git repositories, each with a dedicated CI/CD pipeline:

- **flask-app**: The REST API backend managing authentication, station availability, ML predictions, and AI chat logic.
- **react-app**: A TypeScript-based single-page application (SPA) providing a dynamic and responsive user experience.
- **scraper**: A standalone data ingestion engine for JCDecaux and OpenWeather APIs.

The system is containerized via Docker and deployed on AWS EC2, utilizing AWS RDS (MySQL) for centralized, high-performance data persistence, ensuring that long-running ingestion tasks do not block API responses.

1.2 Target Users and Personas Based on the requirements elicitation process in Sprint 1, we identified three core personas to guide our feature development:

- **Eoin (The Daily Student Commuter):** A 20-year-old computer science student at UCD. He relies on cycling to avoid bus fares and the risk of personal bike theft.
    - **Pain Points:** Arriving at empty stations, finding full stations near campus, and getting caught in unforeseen rain.
    - **Solution:** Real-time map status, integrated weather for "go/no-go" decisions, and a **Machine Learning Prediction model** to confirm availability 10 minutes before departure.
- **Sarah (The Punctual Office Worker):** A 29-year-old marketing manager commuting to the Silicon Docks. She takes the train into the city and uses bikes for the crucial "last mile."
    - **Pain Points:** Uncertainty about route efficiency and historically busy periods.
    - **Solution:** The **Journey Planner** for optimal "walk-cycle-walk" routing and **Historical Data** charts to analyze weekly occupancy trends.
- **Matteo (The Weekend Tourist):** A 35-year-old visitor from Italy who wants to explore landmarks but is unfamiliar with Dublin's layout and rental rules.
    - **Pain Points:** Getting lost and confusion over system mechanics (e.g., pricing, card requirements).
    - **Solution:** Intuitive Map Visualization (with GPS Locate Me) and a **Generative AI Chatbot** for instant, natural language assistance.

1.3 Core Functional Features

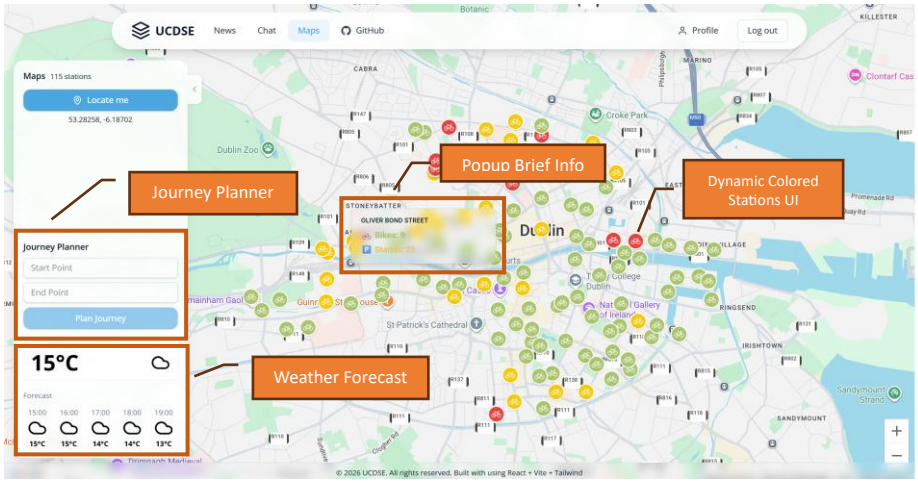


Figure 1.1: Main interactive dashboard with dynamic station status and real-time weather integration.

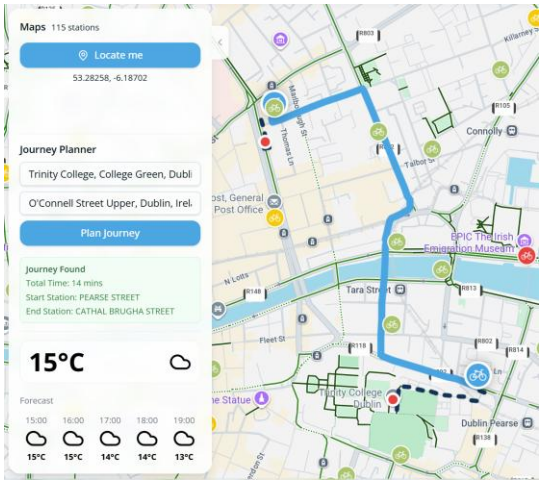


Figure 1.2: Journey Planner with optimal walk-cycle-walk routing.

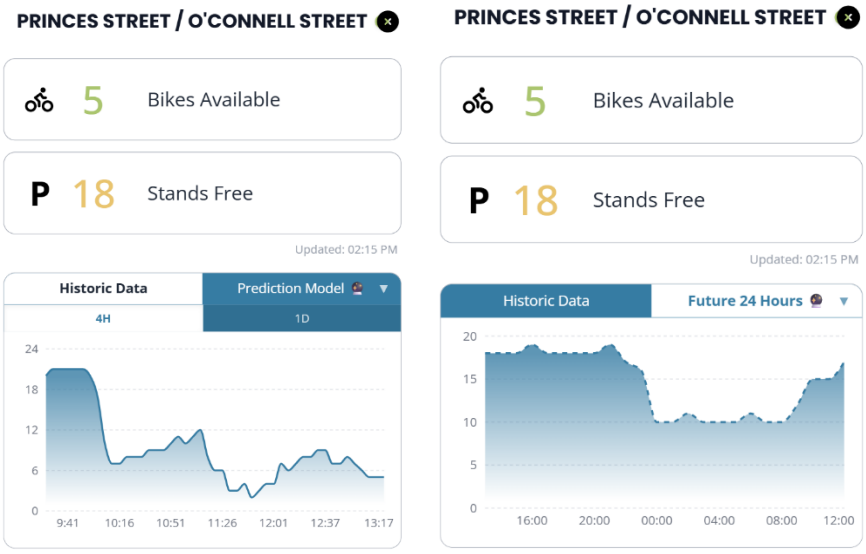


Figure 1.3: ML-driven 24-hour bike availability forecast for a selected station.

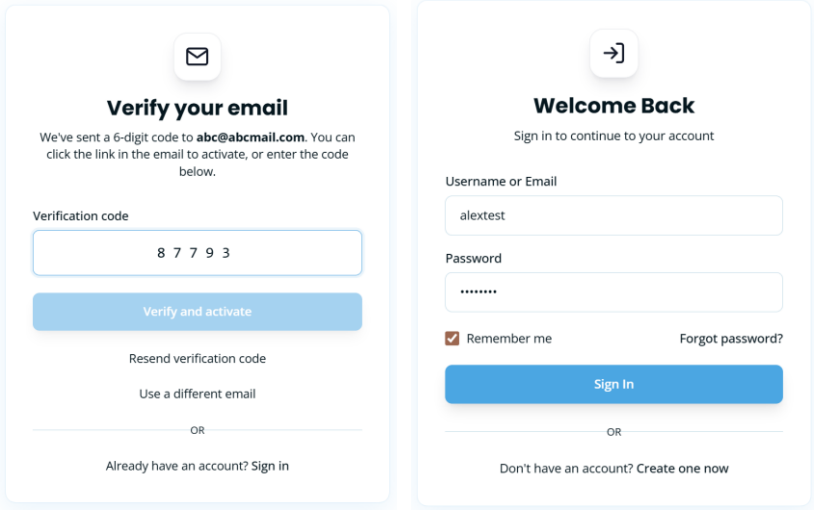


Figure 1.4: Secure JWT-based authentication and account activation interface.

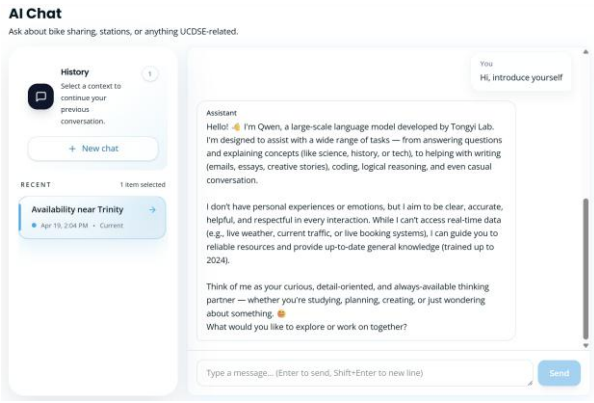


Figure 1.5: LLM-powered assistant.

A. Interactive Map Visualization & Real-Time Data Ingestion

The interactive dashboard serves as the user's primary entry point, providing a live snapshot of the Dublin Bikes network **(See Figure 1.1)**. Users can click the "Locate Me" button to instantly center the map on their position. To help users quickly spot available bikes at a glance, the map employs an intuitive color-coding system (green for >5 bikes, yellow for ≤5, and red for empty stations). Alongside the map, a real-time weather widget displays current conditions and a 5-hour forecast, helping users make immediate "go/no-go" travel decisions.

To deliver this seamless experience, the frontend is built with React and modern Google Maps Web Components. It efficiently processes live data into a quick-lookup dictionary, allowing station colors to update instantly without UI lag. Behind the scenes, a multi-threaded Python daemon continuously ingests live station metrics and OpenWeatherMap data into AWS RDS. When the map loads, the Flask backend utilizes an optimized SQL query to securely and rapidly serve the latest network status to the user.

B. Comprehensive Station Dashboard & Predictive Analytics

When a user clicks on any station marker, a detailed pop-up window opens to display the exact number of free bikes and empty parking stands **(See Figure 1.3)**. By exploring the dual-tabbed interactive chart, users can seamlessly toggle between "Historic Data" to review recent occupancy trends, and the "Prediction Model" to foresee if bikes will be available later in the day.

This rich interactivity is engineered for high performance. Switching between historical timeframes happens with zero network lag because the data is filtered locally on the user's browser using React's useMemo hook. Conversely, requesting a future forecast dynamically triggers the backend's globally cached Scikit-Learn Decision Tree model. The model dynamically calculates predictions by integrating upcoming weather forecasts with time-based features. Furthermore, strict backend logic enforces boundary limits to ensure that predicted bike counts are always mathematically realistic (never dropping below zero or exceeding station capacity), guaranteeing a reliable experience for the user.

1.4 Advanced Optional Features

A. Global Minimum Duration Journey Planner

The Journey Planner is a multi-modal routing engine that recommends the most time-efficient "walk-cycle-walk" commute by calculating the global minimum travel time **(See Figure 1.2)**. To ensure maximum backend performance, the routing engine employs an optimized, multi-stage algorithm:

- **Efficient Data Pre-fetching:** Executes a func.max SQL subquery to retrieve only the latest availability, strictly filtering out stale or non-operational station data without N+1 query overhead.
- **Pruning & Matrix Resolution:** Calculates linear distances to instantly isolate the top 10 viable stations. It then utilizes the Google Maps Distance Matrix API to refine this to the top 5 candidates based on precise walking durations.
- **Global Minimum Calculation:** Evaluates a 5x5 matrix of all 25 valid permutations to pinpoint

the absolute lowest sum of *Walking 1 + Cycling + Walking 2*.

- **Fault Tolerance:** Fortified with a custom @gmaps_retry decorator and explicit timeouts, the engine gracefully handles API quota limits or network interruptions by assigning infinite weights to failed routes, ensuring continuous system stability.

C. Secure Authentication and Session Management

The system features an enterprise-grade security layer utilizing JWT (Access/Refresh Tokens) and 6-digit email verification **(See Figure 1.4)**. Authentication states are isolated in the user and sessions tables of the AWS RDS instance, ensuring secure access to personal chat histories and profile settings.

D. Stateful Generative AI Support

To assist users with natural language queries—such as understanding rental rules or finding landmarks—the application features a highly responsive AI chatbot **(See Figure 1.5)**. Users enjoy fluid, real-time interactions with a live-typing effect and can resume sessions from their personal history. To maintain organization, the system automatically generates context-aware titles for new chats.
Powered by LangChain and Aliyun Qwen-plus, the assistant delivers low-latency responses via Server-Sent Events (SSE). Conversation context is securely archived using SQLChatMessageHistory, optimizing performance through SQLAlchemy connection pooling. For security, the backend generates SHA-256 hashed session IDs and strictly verifies user ownership. It also handles concurrent writes via IntegrityError rollbacks and utilizes a secondary LLM invocation for automated topic classification.

2. Requirements

2.1 Requirements Elicitation Process

To ensure the Dublin Bikes application delivers meaningful value to its diverse user base, we implemented a structured, user-centric requirements elicitation process during Sprint 1. The goal was to bridge the gap between high-level project objectives and concrete technical specifications through empirical research and stakeholder analysis.
The process followed four distinct phases:

- **Market Research & Competitive Analysis:** We conducted a benchmarking study against existing bike-sharing platforms such as the official TFI Bikes app, Villo! (Brussels), and Santander Cycles (London). This research identified a significant market gap in Dublin: the lack of integrated predictive availability and real-time weather-sensitive routing.
- **Qualitative User Interviews:** To understand local pain points, we conducted structured interviews with three primary stakeholders. These sessions uncovered distinct needs: Eoin's anxiety over bike theft and sudden rain, Sarah's need for "last-mile" reliability from the train station, and Matteo's confusion regarding rental rules.
- **Persona Synthesis:** Insights from the interviews were synthesized into three distinct personas: Eoin (the budget-conscious student), Sarah (the punctual professional), and Matteo (the weekend tourist). These personas served as the "living documentation" that guided every design decision.

- **User Story Mapping:** Finally, we translated user needs into formal User Stories and Acceptance Criteria (AC). This provided us with a clear "Definition of Done" for each feature and formed the basis of our automated testing suite.

2.2 App Mockups and Design Rationale

The following mockups represent the most critical interfaces of the application, designed to address the specific needs identified during the elicitation process.

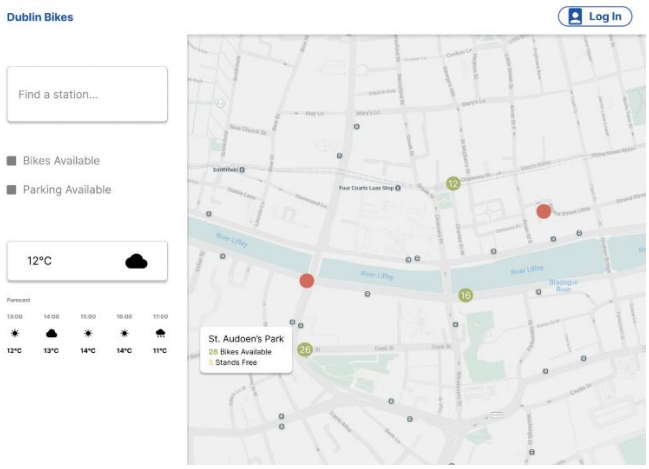


Figure 2.1: Primary dashboard featuring the Google Maps SDK integration and weather widget.

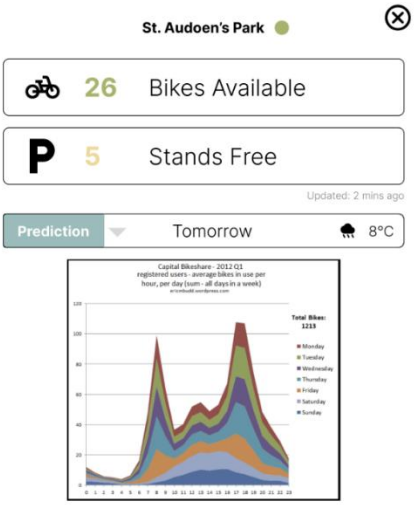


Figure 2.2: Detailed station view showing the toggle between Historical Trends and ML Predictions.

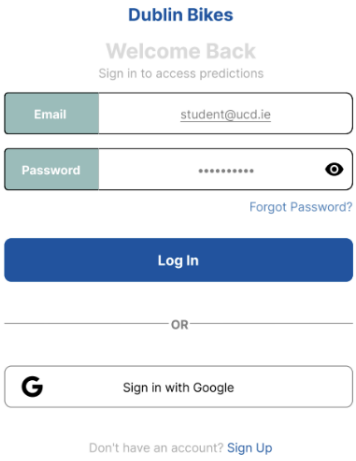


Figure 2.3: User login interface.

Figure 2.1: Primary Dashboard & Weather Widget

Design Focus: This interface addresses Eoin's need for a "go/no-go" decision and Matteo's spatial navigation. By pairing a full-screen map with interactive hover states, a "Locate Me" GPS function, and a 5-hour weather forecast, it establishes immediate foundational situational awareness.

Figure 2.2: Station Details & ML Predictions

Design Focus: This dual-tabbed dashboard directly serves our commuter personas. Sarah utilizes the "Historic Data" tab to analyze long-term occupancy trends (e.g., Monday mornings at the train station), while Eoin relies on the 24-hour "Prediction Model" to preemptively confirm bike availability 10 minutes before leaving his apartment.

Figure 2.3: User Authentication Interface

Design Focus: This interface provides a secure user authentication process. By facilitating simple registration and login workflows, it ensures that frequent users like Sarah and Eoin can safely manage personal accounts and access customized features, such as saved AI chat history.

2.3 User Stories and Acceptance Criteria

The following table outlines the core functional requirements that drove the development of the Dublin Bikes ML App.

User Story ID	User Story Description	Acceptance Criteria (AC)
US-01	As a student running late (Eoin), I want to predict if bikes will be available in 10 minutes, so that I don't walk to an empty station.	**AC1 (ML Prediction):** **Given** the user has opened a station's detailed popup, **When** the user selects the "Prediction Model" tab, **Then** the system renders a 24-hour ML-driven forecast. **AC2 (Data Boundary):** **Given** the system is generating the prediction chart, **When** the model returns the values, **Then** the displayed counts must be mathematically clamped between 0 and total capacity.
US-02	As a commuter planning ahead (Sarah), I want to view historical occupancy trends, so that I know which days are typically busy.	**AC1 (Historical Chart):** **Given** the user is viewing the station details, **When** the user selects the "Historic Data" tab, **Then** the system renders a line chart of past availability. **AC2 (Time Filtering):** **Given** the historic data chart is displayed, **When** the user toggles between "4H" and "1D" ranges, **Then** the UI instantly filters the data locally without triggering a new network request.
US-03	As a time-sensitive professional (Sarah), I want to use a journey planner to find the fastest route, so that I arrive at my meetings efficiently.	**AC1 (Calculate Route):** **Given** the user is interacting with the Journey Planner widget, **When** the user submits valid start and end points, **Then** the system calculates the optimal "Walk-Cycle-Walk" combination to minimize total time. **AC2 (Display Route):** **Given** the system has successfully calculated the optimal route, **When** the result is returned to the frontend, **Then** the map renders a polyline path and displays

		the total estimated time of arrival (ETA).
US-04	As a tourist (Matteo), I want to ask questions using natural language via an AI assistant, so that I can understand rental rules easily.	**AC1 (Process Query):** **Given** the user is on the AI Chat interface, **When** the user submits a natural language query, **Then** the system processes it via the LangChain/LLM and returns a context-aware response. **AC2 (Persist Context):** **Given** an authenticated user is using the AI Chat, **When** a conversation occurs, **Then** the system persists the chat interactions in the database to maintain session history.
US-05	As a frequent user, I want to register and log in securely, so that I can access personalized features like saved routes and AI chat.	**AC1 (Email Verification):** **Given** the user is registering a new account, **When** the registration form is submitted, **Then** the system emails a 6-digit verification code to activate the account. **AC2 (Secure Authentication):** **Given** the user is on the login interface, **When** the user enters valid and verified credentials, **Then** the system authenticates the user by issuing secure JWT (Access/Refresh) tokens.

2.4 Supplementary Material

To maintain a concise report, the extensive raw data generated during the requirements phase has been archived. This includes:

- Full Interview Transcripts: Comprehensive logs from sessions with Sarah, Matteo, and Eoin.
- Detailed Persona Profiles: In-depth analysis of user demographics, goals, and frustrations.
- Competitive Analysis Matrix: A technical feature comparison across major European bike-sharing systems.

Reference Note: All aforementioned comprehensive documentation related to our elicitation process can be accessed in our GitHub repository at: **https://github.com/ucdse/docs/tree/main/requirements**

3 Architecture and Design

3.1 Overall Architecture

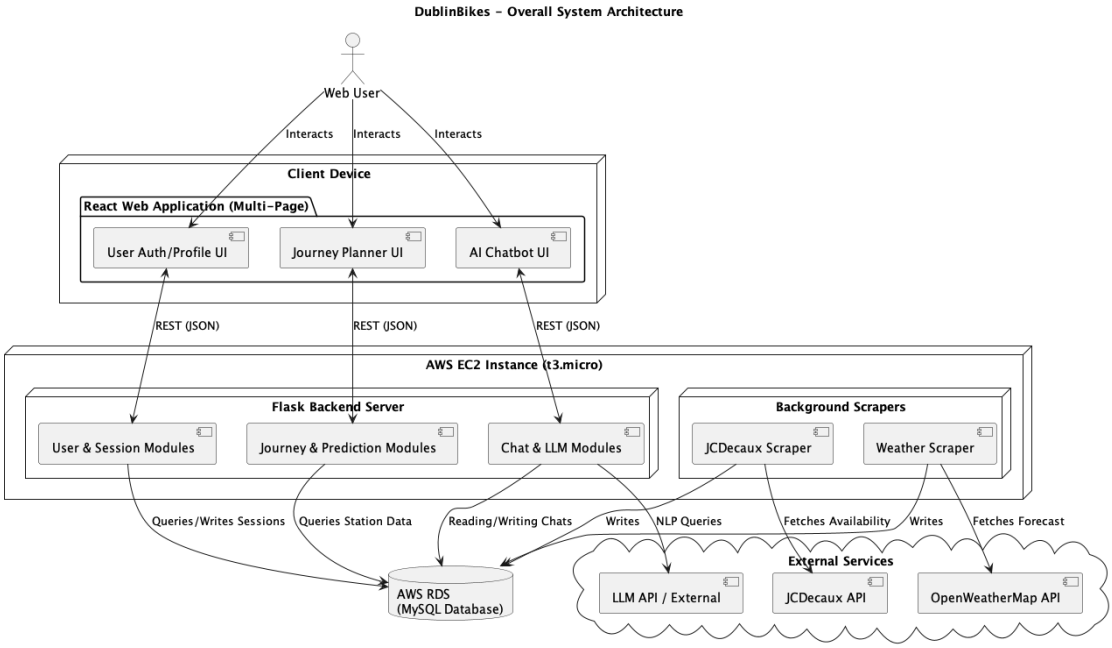


Figure 3.1 Overall System Architecture

Our project utilizes a decoupled client-server and cloud architecture, separating the front-end user interfaces, back-end web routing, and the data acquisition processes.

The architecture (Figure 3.1) illustrates a scalable Web application split into three robust functions: user authentication, journey planning, and an AI chatbot.

Client Device: A React-based multi-page application (MPA) utilising distinct route pages (/login, /maps, /chat). It communicates to the backend asynchronously via JSON REST APIs seamlessly across the layout.

Server Environment (AWS EC2): A scalable AWS EC2 micro instance hosts our core Python layer.

Flask Backend Server: Acts as the traffic controller, routing API requests, formatting data, and calling our predictive models.

Background Scrapers: Standalone scripts continuously gather live data from the JCDecaux API and OpenWeatherMap API, pushing raw data into our database independently of the Web Server.

Database Layer (AWS RDS): A managed, centralised AWS RDS MySQL instance that securely maintains static station references, dynamic availability histories, and weather data independent of the EC2 lifecycle.

This separation of concerns guarantees that long-running data-fetching tasks don't block API responses, and ensures that the client remains highly responsive.

3.2 Class Diagram

3.2.1 Backend Domain & Services

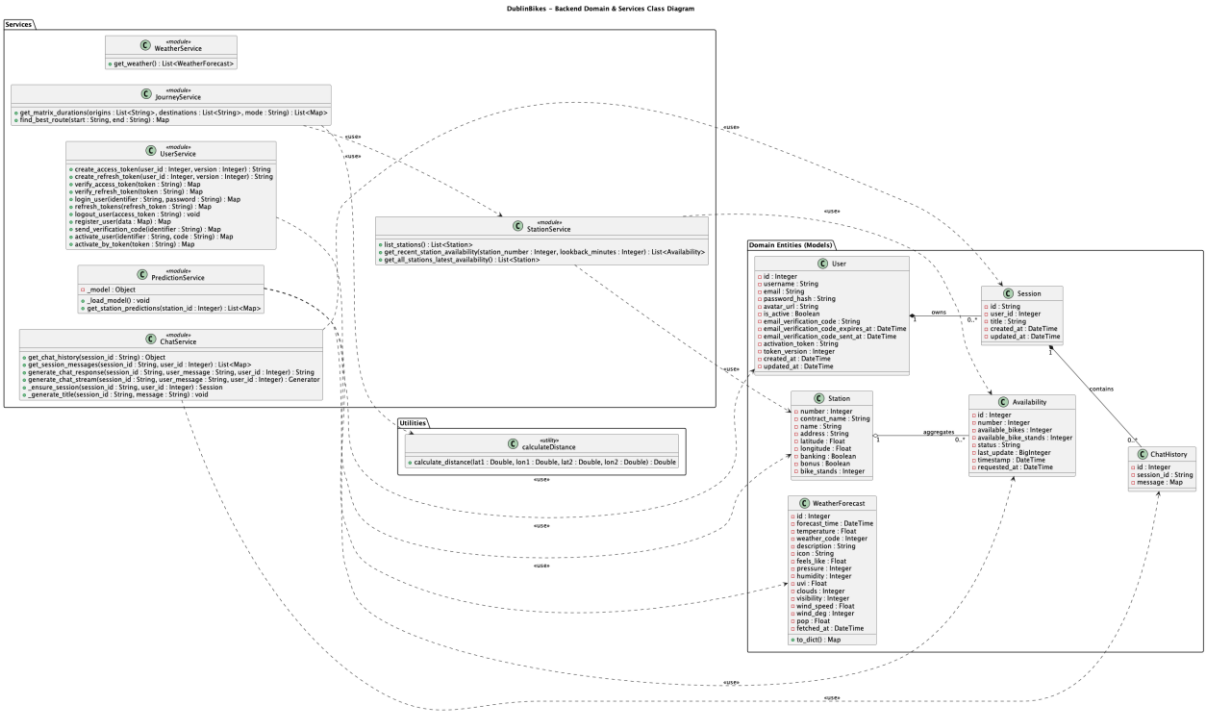


Figure 3.2 Backend Class Diagram

The backend class diagram (Figure 3.2) provides a deep architectural view of our core Python layer, explicitly distinguishing between Domain Entities and internal software Service classes. This duality is fundamental to expressing the core business logic of the system.

The Domain Entities (such as “Station”, “Availability”, and “User”) are designed to capture the central concepts within our problem domain. We utilize strict structural associations, depicted by solid lines, to define the long-term, static relationships between these objects. For instance, the relationship between a “User” and a “Session” is modeled as a composition. It semantically dictates that a session structurally belongs to a specific user, and its lifecycle is rigidly constrained by the user's existence. Conversely, the relationship between a “Station” and its “Availability” records is modeled as an aggregation. This signifies a "part-whole" dynamic where physical stations and their highly volatile, real-time availability metrics are inherently connected, yet a station entity logically retains its independence regardless of specific, transient availability records.

Operating above these entities is the service layer. Service classes (such as “JourneyService” and “StationService”) reside purely within the software. Their primary responsibility is to orchestrate complex behaviors and algorithms that do not naturally belong to any single domain entity. To model the interactions within this layer, we made a deliberate design decision to extensively employ use dependencies.

This design choice is crucial for demonstrating the underlying collaborative logic of the application.

Taking the “JourneyService” as a prime example: it does not structurally own any station data. Its core mandate is to execute sophisticated pathfinding algorithms. However, to successfully complete a behavior like “find_best_route()”, it must temporarily invoke the “StationService” to acquire the latest real-time station statuses. By mapping this as a dashed dependency rather than a solid structural association, we explicitly enforce the principle of separation of concerns. It illustrates a short-term, behavior-driven collaboration rather than structural ownership, thereby effectively minimizing tight coupling between independent software modules.

3.2.2 Frontend React Components

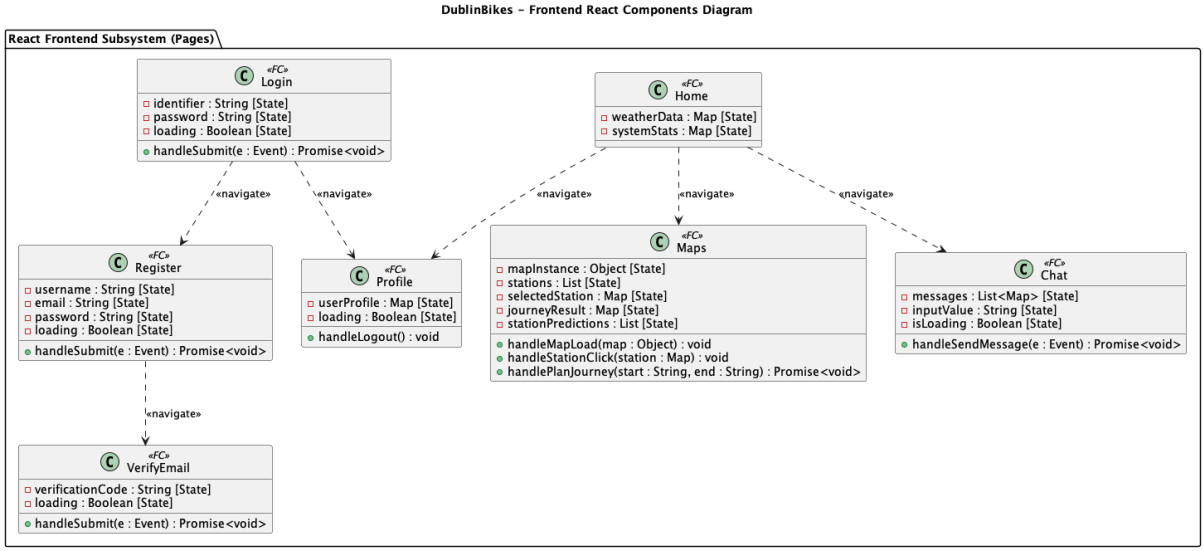


Figure 3.3 Frontend Class Diagram

Rather than visualizing (Figure 3.3) the frontend as an ambiguous monolith, each core page (such as “Maps” or “Chat”) is represented as an isolated, self-contained functional component. These components strictly encapsulate their local React states as private properties (for example, “-mapInstance [State]” within the mapping component) while explicitly declaring their core event handlers (e.g., “handleSubmit”, “handlePlanJourney”) as public methods.

Furthermore, to cleanly map the user's operational flow throughout the web application, we utilized “navigate” dependency arrows between these components. This approach provides a clear, high-level blueprint of how a user transitions from the landing dashboard into deeper application features, entirely avoiding the anti-pattern of cluttering the class diagram with HTTP routing mechanics or network implementation details.

3.3 Sequence Diagrams of Core Use Cases

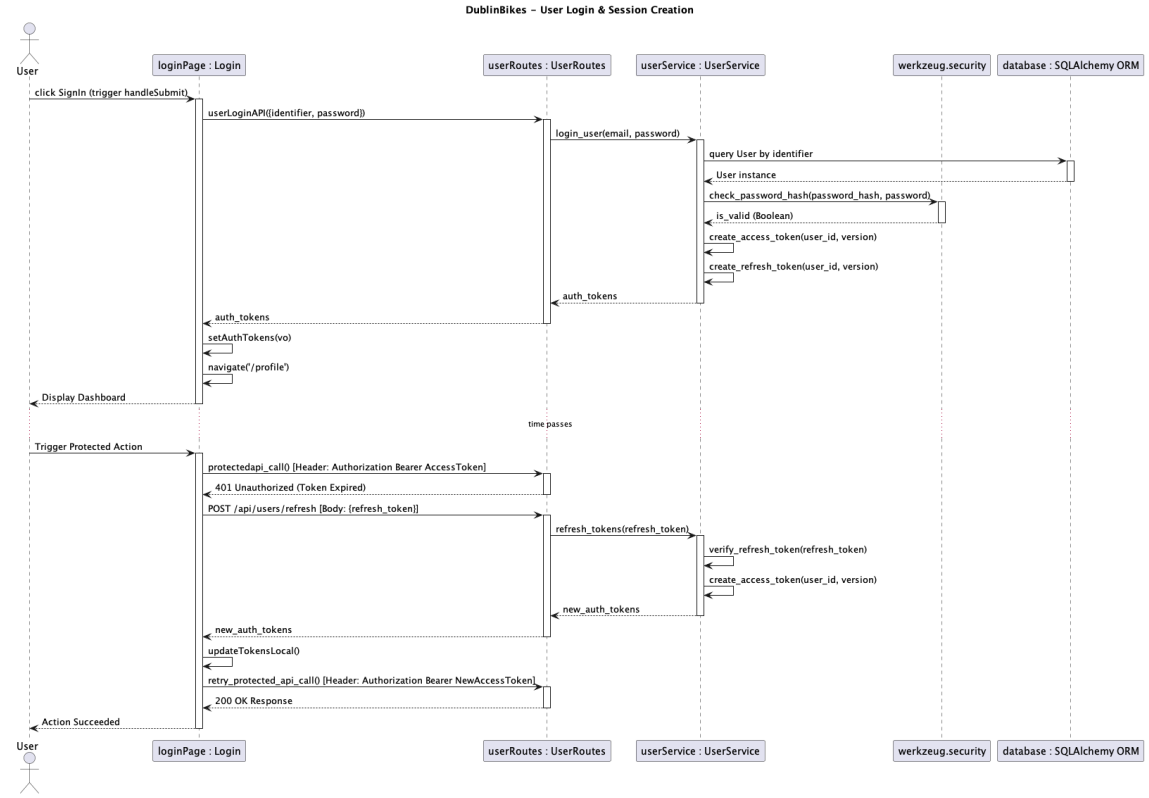


Figure 3.4 User Login and Session Creation

User Login (Figure 3.4): Details the flow starting from the UI instance (“loginPage: Login”) capturing user input via “handleSubmit()”. The payload routes through “userService” which executes “login_user()”, performs an inline database query to retrieve the user record, verifies passwords using the “werkzeug.security” library, and concludes with the UI instance calling “setAuthTokens()” and “navigate('/profile')”.

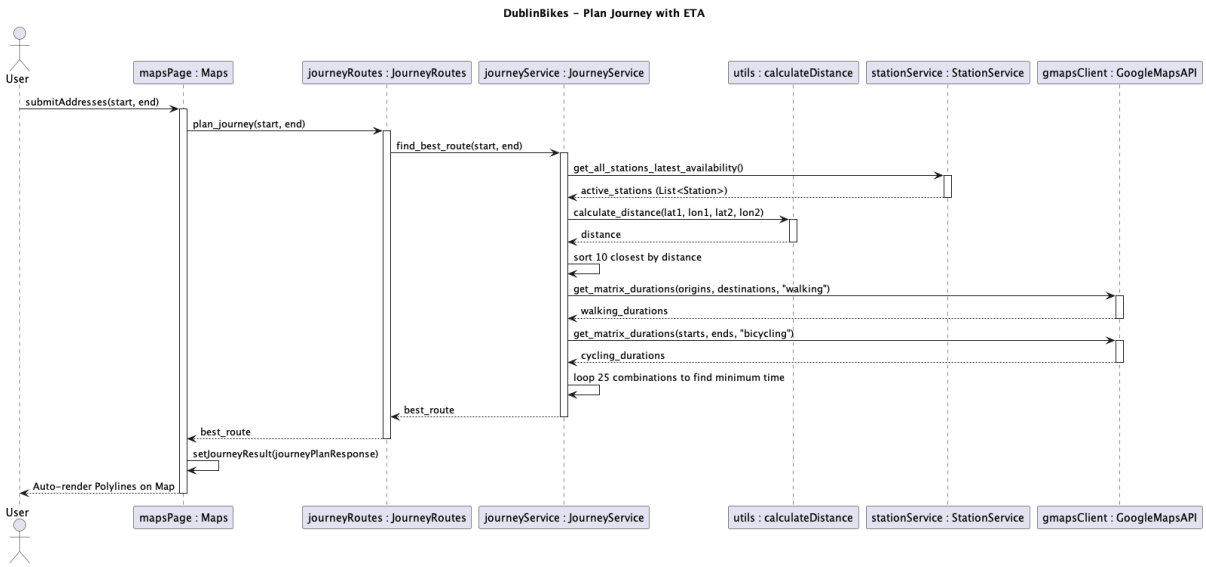


Figure 3.5 Journey Planner

Journey Planner (Figure 3.5): Initiated from “mapsPage: Maps”, the “journeyService” instance coordinates complex routing. It leverages the “stationService” to fetch active stations, executes inline algorithmic self-messages to filter top stations via Haversine distance (“calculate_disrance”) and evaluate 25 matrix combinations for the global minimum time, and performs matrix computations against “gmapsClient: GoogleMapsAPI”. Finally, the UI instance calls “setJourneyResult()” to declaratively render the map polylines.

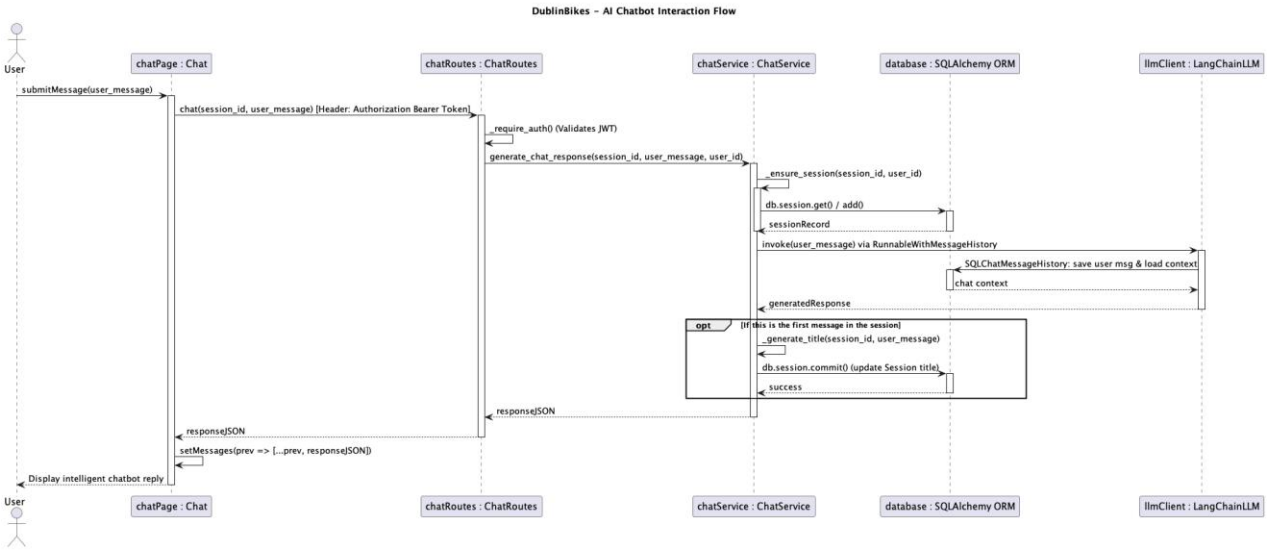


Figure 3.6 AI Chatbot

AI Chatbot (Figure 3.6): The user triggers “submitMessage()”. The API explicitly validates identity by parsing the “Authorization Bearer” HTTP header via “_require_auth()”.The “chatService” executes inline SQLAlchemy database operations to ensure session continuity, and then passes context to the “llmClient: LangChainLLM” via “invoke()”, which internally utilizes “SQLChatMessageHistory” to seamlessly save and load dialogue state, ending with the UI component updating state via “setMessages()”.

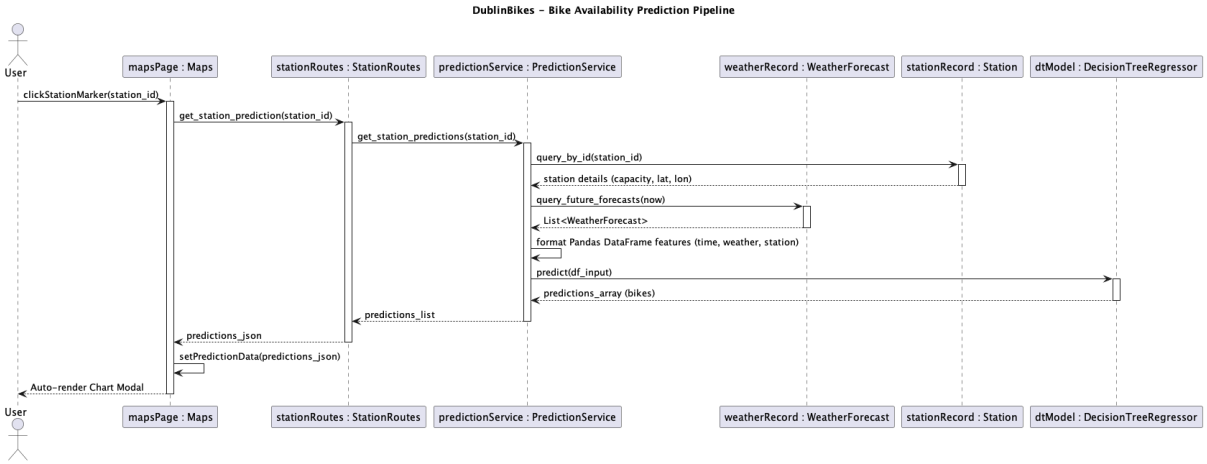


Figure 3.7 Bike Availability Prediction

Bike Availability Prediction (Figure 3.7): Outlines the Machine Learning inference loop triggered by "clickStationMarker()". The "predictionService" queries the "stationRecord: Station" for static capacities and the "weatherRecord: WeatherForecast" for future meteorological data. It executes an inline process to format a Pandas dataframe feature array (combining time, weather, and station metrics) before passing it to the "dtModel: DecisionTreeRegressor" instance's "predict()" method, concluding with the UI updating state via "setPredictionData ()" to declaratively render the chart.

To address typical initial cold-start latencies inherent to ML modelling, the "PredictionModel" referenced in the prediction sequence is pre-loaded directly into Flask's memory upon initialization.

3.4 Database Design and Design Choices

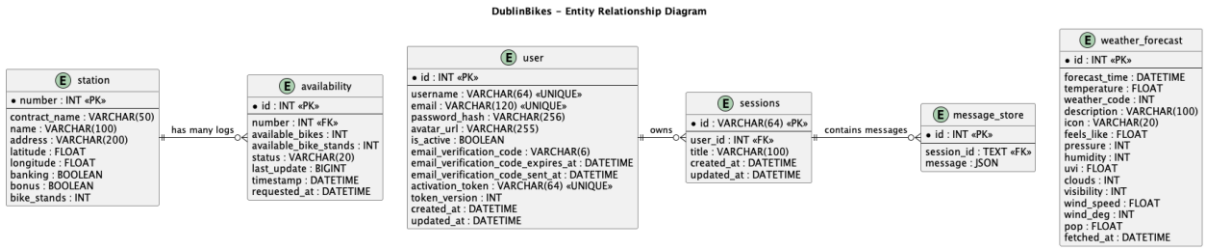


Figure 3.8 Entity Relationship

Managed AWS RDS (Relational Cloud SQL): We opted for a cloud-managed Relational Database (AWS RDS for MySQL) rather than a local SQLite file or NoSQL. Shared bike systems and meteorological data are highly structured datasets. Given that the Machine Learning modelling pipeline requires performing time-series transformations and heavy "JOIN" operations across station IDs and timestamps, a robust cloud SQL environment was naturally the correct fit.

Data normalization (Figure 3.8): Static factors like a station's latitude, longitude, and internal naming ("station") remain constant. Instead of redundantly saving these details with every availability check, they are separated. The "availability" records highly volatile factors (every 5-10 minutes) leveraging Foreign Keys mapping back to the static tables, reducing storage footprint and read redundancy.

Indexing (Figure 3.8): Because querying time-aligned weather logs and bike availability histories spans millions of rows, we introduced specific database indexing on the "timestamp" and "number" columns. This design drastically diminished query delays when compiling features to refresh our predictive decision tree classifiers.

Stateful LLM persistence & auth isolation (Figure 3.8):: To securely expand the application beyond public bike metrics, we isolated authentication and Chatbot states via the "user", "sessions", and "message_store" tables. The "user" records enforce robust identity and JWT validations, mapped via foreign key explicitly to Chatbot metadata ("sessions"). To accommodate the vast, unpredictable text structure characteristic of Large Language Models via LangChain, the "message_store" table purposefully archives deep ongoing chat interactions leveraging native JSON columns.

3.5 Additional Material

All original UML files are securely tracked in our GitHub repository **https://github.com/ucdse/docs/tree/main/UML_files.**

4. Machine Learning Model

This chapter describes the machine-learning component of the Dublin Bikes project, whose purpose is to predict the number of available bikes at any given station for every weather-forecast entry currently cached in the database. In the current implementation, only forecast rows up to 48 hours ahead of the current UTC hour are stored. The complete training and evaluation code is version-controlled in the main repository under flask-app/machine_learning/ml.ipynb, together with the exported artefacts bike_availability_model.pkl and model_features.pkl.

4.1 Problem Definition and Target Variable

The business objective is to help users decide when and where to pick up a bike by forecasting future availability. Consequently, the problem is framed as a supervised regression task:

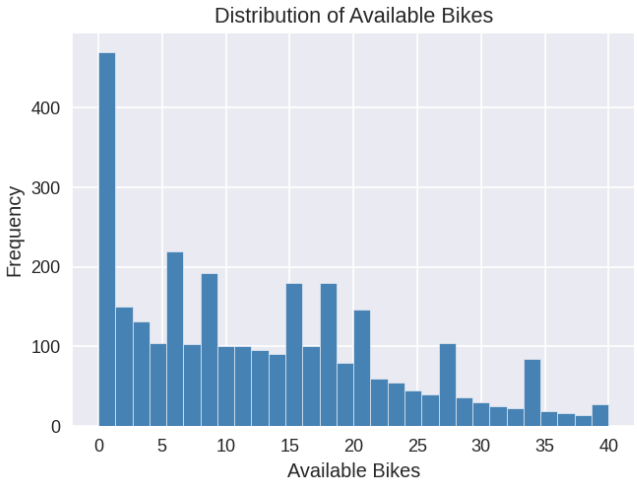
- **Target variable:** num_bikes_available – the number of bikes that can be rented from a station at a specific point in time.
- **Inputs:** A vector of temporal, spatial, and meteorological features that are known in advance or can be observed at prediction time.

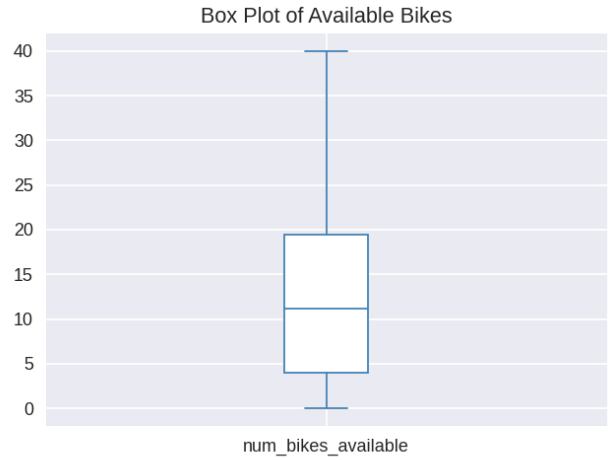
Predictions are produced in bulk for every forecast entry stored in the weather_forecast table whose forecast_time is at or after the current hour, and the results are exposed to the frontend via the GET /api/stations/<number>/prediction endpoint.

4.2 Dataset and Data-Cleaning Process

4.2.1 Raw Data

The raw training set is the file final_merged_data.csv. It contains 298,946 rows and 78 columns, built by joining historical station-availability snapshots with historical weather readings provided by Met Éireann. Each row represents the state of one specific station at one specific hour.





4.2.2 Data-Cleaning and Feature-Selection Rationale

Category of removed columns	Reason
All *_quality_indicator columns	Metadata flags with no meteorological signal.

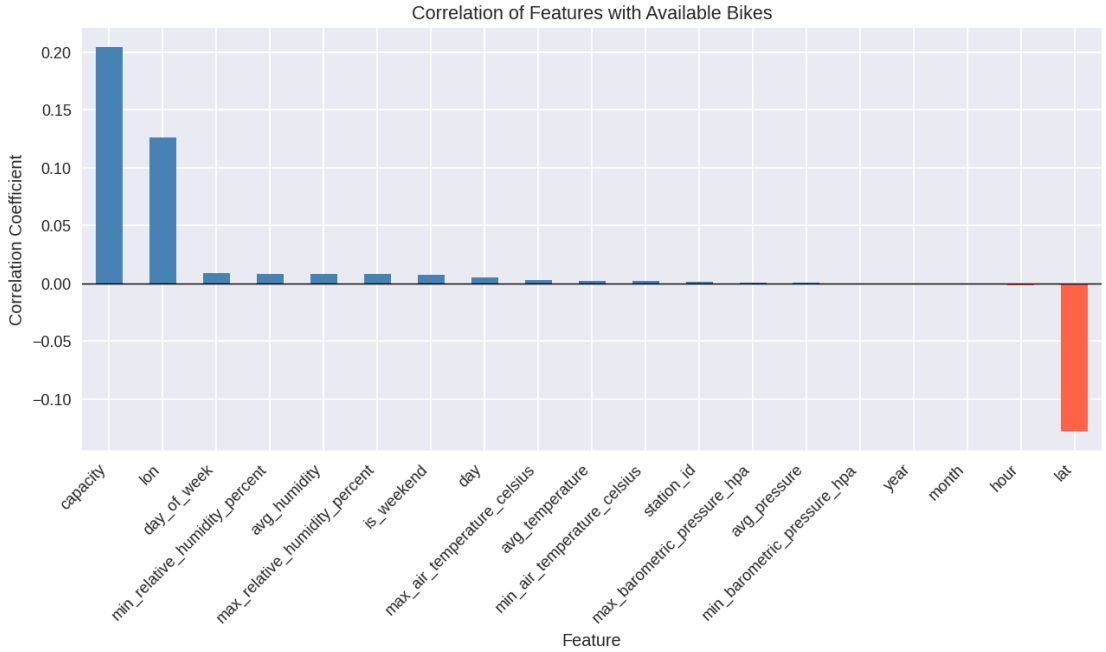
Category of removed columns	Reason
All *_std_deviation columns	Redundant once corresponding min/max or mean values are retained.
Soil / grass / earth temperature columns	Low correlation with urban bike-usage patterns.
Text columns (name, address, last_reported)	Not numerical; location identity encoded by station_id, lat, and lon.
Near-constant booleans	Almost 100% True; providing no discriminative power.

4.2.3 Feature Engineering

Five derived features were created to capture periodic and meteorological patterns:

- day_of_week, is_weekend, avg_temperature, avg_humidity, avg_pressure.

Additionally, the columns month and year were removed during the correlation analysis phase because the training data covers only December 2024, meaning both columns carry a single constant value (zero variance) and provide no discriminative signal to the model. The final feature set used for model training consists of eleven features: station_id, capacity, lat, lon, hour, day, day_of_week, is_weekend, avg_temperature, avg_humidity, avg_pressure.



4.3 Training and Testing Process

The dataset was split using a 70% training / 30% testing ratio (random_state=42). Four candidates were evaluated: Linear Regression, Decision Tree Regressor, Random Forest Regressor, and Gradient Boosting Regressor. The Decision Tree was trained with default scikit-learn parameters (no maximum depth constraint, no minimum sample restrictions), allowing the tree to grow fully. The Random Forest was trained with n_estimators=100, max_depth=15, and min_samples_leaf=10 to limit tree depth and reduce variance. Both the Gradient Boosting and Linear Regression models were trained with their respective

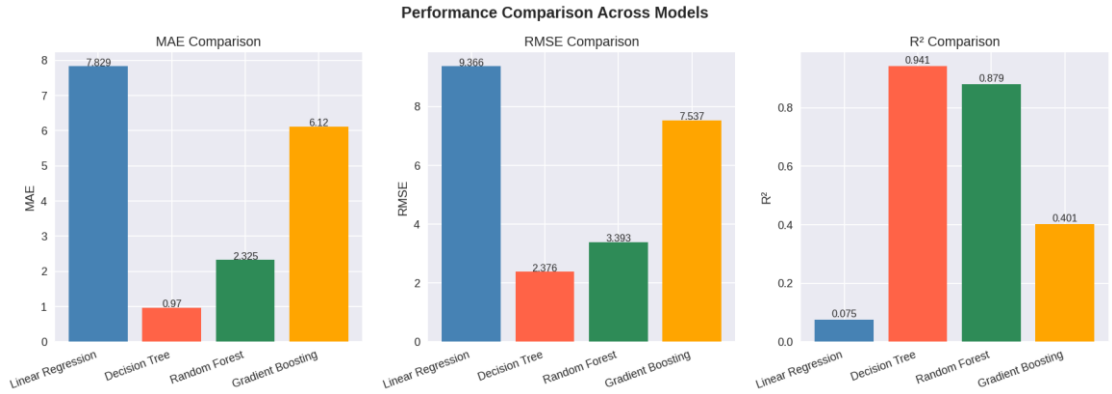
default parameters.

4.4 Results

4.4.1 Model Comparison

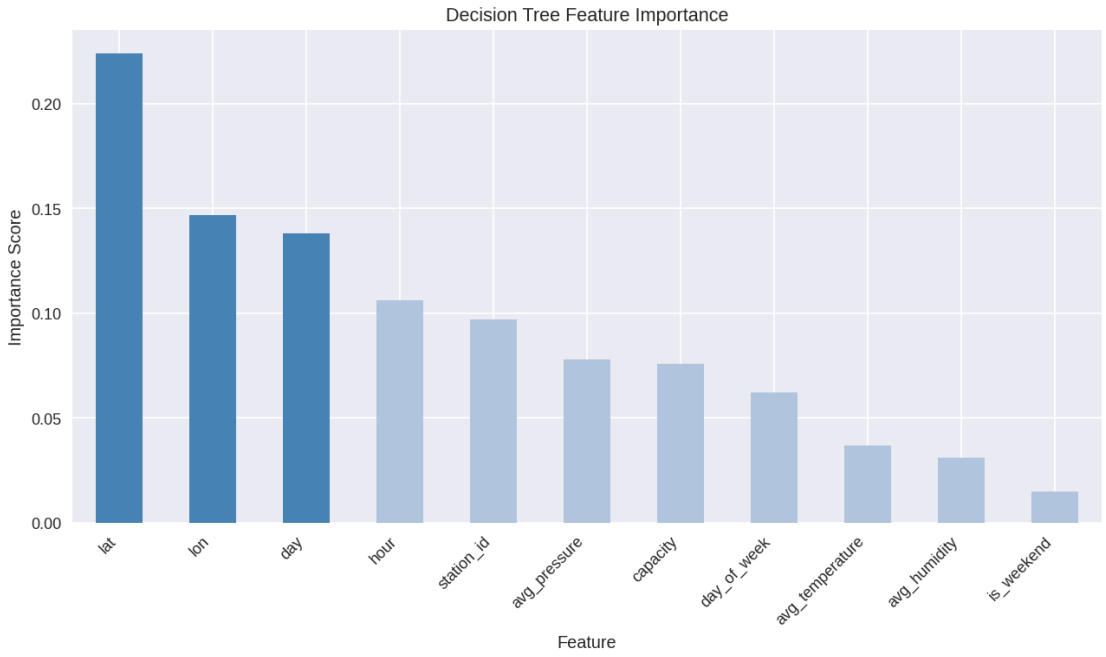
Model	MAE	RMSE	R^2
Linear Regression	7.829	9.366	0.075
Decision Tree	**0.970**	**2.376**	**0.941**
Random Forest	2.325	3.393	0.879
Gradient Boosting	6.120	7.537	0.401

Based on the superior performance metrics, the Decision Tree was selected as the final model for production deployment.



4.4.2 Feature Importance Analysis

The selected Decision Tree model was used to inspect feature importance, identifying which features contribute most to predicting bike availability. Geographic features (lat, lon) and temporal features (day, hour) ranked highest, followed by avg_pressure, station_id, and capacity. Weather-derived averages (avg_temperature, avg_humidity) and is_weekend contributed the least.



4.5 Reflection and Future Work

4.5.1 Strengths

- **Straightforward Pipeline:** No missing-value imputation is needed, and the feature set is compact, leading to fast retraining.
- **Actionable Predictions:** The output is a direct bike count, making it user-friendly for the frontend.
- **Decoupled Architecture:** The REST endpoint allows independent updates to the model and the UI.

4.5.2 Limitations

- **Temporal Scope:** Training data covers only December 2024. Seasonal variations are not captured, potentially reducing accuracy in other months.
- **Weather-source Mismatch:** Difference between Met Éireann daily aggregates (training) and OpenWeatherMap hourly forecasts (inference) introduces distributional shift.
- **Overfitting Risk:** While the Decision Tree achieves exceptional metrics (MAE 0.970, R^2 0.941), it is inherently prone to overfitting. Performance on unseen data might be less stable without further regularization.

4.5.3 Future Improvements

- **Data Expansion:** Incorporate multiple seasons to improve generalization.
- **Lag Features:** Add historical availability (e.g., t-1 hour) to exploit temporal autocorrelation.
- **Hyperparameter Optimization:** Perform systematic grid search or Bayesian optimization on the Decision Tree (e.g., tuning max_depth, min_samples_leaf) to prevent overfitting.
- **Uncertainty Communication:** Introduce prediction confidence intervals for users.

5. Testing and Quality Assurance

5.1 Strategy and Coverage Results To ensure the reliability, security, and performance of the Dublin Bikes backend architecture, we implemented a robust automated testing strategy. Built using **Pytest** (v9.0.3) and pytest-cov, our approach encompassed both isolated Unit Tests and end-to-end Integration Tests for the Flask API layer.

To ensure hermetic testing environments, we configured conftest.py to route all test operations to an isolated in-memory SQLite database (sqlite:///:memory:). This guaranteed fast execution and prevented any pollution of the production MySQL instance. The suite executed **355 test cases** in just **13.04 seconds**, achieving an exceptional **97% Statement Coverage** across 1,272 lines of code.

Continuous Integration via Jenkins: This Pytest suite is seamlessly integrated into our **Jenkins CI/CD pipeline** running on a Kubernetes Agent. As defined in our project configuration, upon code commits, the pipeline automatically executes Python syntax checks, runs the entire test suite, downloads the prediction models from Hugging Face, and builds the Docker image. This strictly enforces quality gates, ensuring that no failing code can be deployed to our AWS EC2 production server.

Coverage Breakdown:

- **API Routing & Data Contracts (app/api/*, app/contracts/*):** 95% - 100%
- **Journey, ML Prediction & Weather Services:** 100%
- **User Authentication Service:** 96%
- **AI Chat Service:** 92%

5.2 Key Testing Scenarios and Methodologies Achieving deterministic results required rigorously testing our core business logic against complex boundary conditions and external dependencies. We structured our tests around the following critical domains:

1. Machine Learning Boundary Clamping (Prediction Service) Because the ML model generates probabilistic outputs, it is critical to ensure the frontend receives mathematically logical data. We implemented explicit boundary tests:

- test_predictions_clamped_to_zero_minimum: Ensures the system never returns a negative number of available bikes.
- test_predictions_clamped_to_bike_stands_maximum: Guarantees predictions never exceed a station's physical dock capacity.
- test_raises_prediction_error_when_no_forecasts: Validates robust error handling when future forecast data is temporarily unavailable.

2. Security and JWT Authentication (User Service) To secure our stateless architecture, the test suite simulated various authorized and unauthorized access attempts using custom auth_headers fixtures:

- test_creates_valid_jwt & test_valid_token_decoded_successfully: Verifies successful token issuance and decoding.
- test_expired_token_raises_auth_error: Enforces strict Time-To-Live (TTL) boundaries, ensuring expired sessions trigger HTTP 401 responses.

- test_wrong_secret_raises_auth_error: Validates that tokens tampered with or signed by unknown keys are correctly rejected.

3. Temporal Data Filtering (Weather Service) Handling time-series weather data requires strict temporal logic. Our tests ensure users only see relevant forecasts:

- test_past_forecasts_excluded: Validates that any historical weather data is stripped from the API payload.
- test_current_uses_earliest_forecast: Ensures the "current weather" widget always defaults to the most immediate upcoming hour.
- test_hourly_items_contain_pop_field: Verifies that the Probability of Precipitation (PoP) data is consistently mapped for frontend "go/no-go" UI indicators.

4. External API Mocking & Journey Planner We extensively utilized unittest.mock.MagicMock to intercept third-party network calls at the module level. This guaranteed 100% test determinism and prevented API quota exhaustion during CI pipeline execution. Key mocking scenarios included:

- test_journey_service_matrix.py (Google Maps Client): Mocked routing APIs to validate our "walk-cycle-walk" multi-segment algorithms offline.
- test_chat_service_llm.py (LangChain LLM): Stubbed AI interactions to verify stateful chat memory and prompt injection logic without incurring live inference latency.
- test_email_utils.py (SMTP Email Configuration): Suppressed external network calls for email verification by utilizing Flask's MAIL_SUPPRESS_SEND=True setting.

5.3 Test Execution Logs The comprehensive and unedited terminal output of our test suite execution, including module-by-module progress, specific test function names, and the final 355 passed metrics, has been exported as a supplementary file. For detailed verification, please refer to the attached **Pytest_Execution_Logs.md** file located in the project repository's root directory.

6. Process

6.1 Project Organisation and Tools

The project is split across three Git repositories, each with its own Jenkins CI/CD pipeline deploying to AWS EC2 via Docker:

Repository	Role
flask-app	REST API (Flask/Gunicorn), JWT auth, ML predictions, journey planning, AI chat. Owns the MySQL schema via Flask-Migrate.
react-app	Single-page frontend (React 19, TypeScript, Vite 7, Tailwind CSS 4). Served by nginx in production.
scraper	Long-running Python process: polls JCDecaux stations and OpenWeatherMap forecasts, writing to the shared database.

Tooling: Git feature-branch workflow • Jenkins (Kubernetes agent) • Docker • AWS EC2 • pytest (≈97 %

coverage) • ESLint / TypeScript • Discord (team communication).

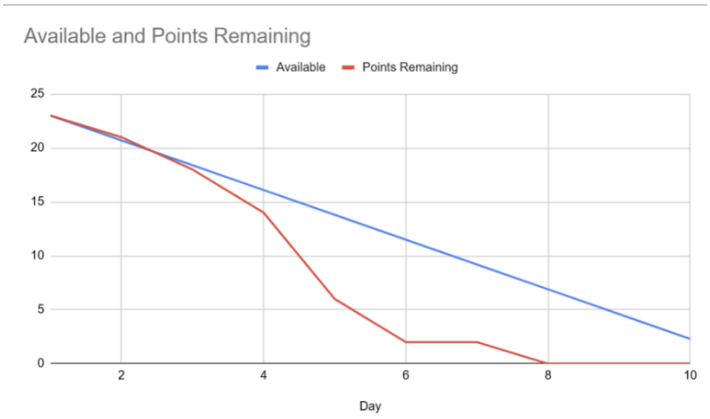
Integration contract: Both flask-app and scraper share one DATABASE_URL. Flask-Migrate migrations must run first. In development, Vite proxies /api to Flask on port 5000; in production a reverse proxy handles the same routing.

6.2 Sprint 1 — Requirements Engineering & Data Collection

Implemented Features and Design Decisions

The team conducted user interviews to validate three personas (Student, Professional, Tourist), produced Figma UI mockups, and defined user stories. On the technical side, a MySQL database schema was established and scraping scripts for the JCDecaux and OpenWeatherMap APIs were written and deployed. A 5-minute scrape interval was chosen as the design default to balance data freshness against API rate limits. The scraper ran stably throughout the sprint, consistently populating the database and providing a reliable foundation for subsequent development.

Burndown Chart



The Burndown chart shows a steep decline in remaining effort. Our "Actual Remaining Effort" line dropped below the "Ideal Trend" line around Day 4, indicating that the team worked faster than anticipated. We reached zero remaining tasks 2 days before the sprint deadline. This high velocity allowed us to dedicate the remaining time to backlog refinement and researching the feasibility of optional features.

Sprint Review

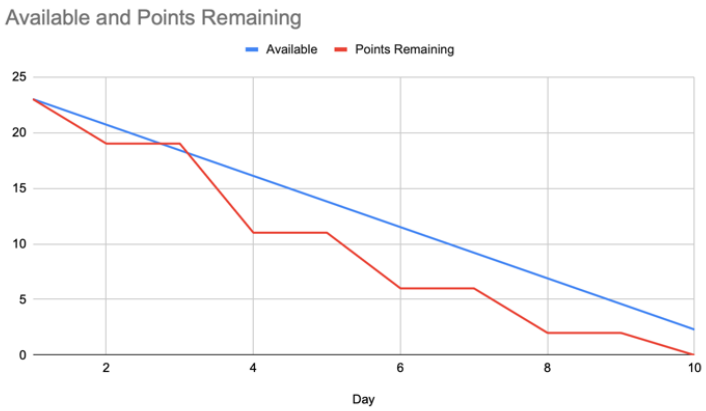
1. **Mockup Walkthrough:** Presented the Figma/PDF mockups, explaining how the UI solves the specific pain points of our personas.
2. **Data Verification:** Showed the live MySQL database tables with rows of real-time data collected over the past few days, proving the stability of our scrapers.
3. **Scope Discussion:** Since we finished early, we raised a discussion regarding the "Smart Journey Planner" feature. We are currently evaluating if adding this complex feature fits within our remaining timeline. We agreed to finalize this decision in the next Sprint Planning.

6.3 Sprint 2 — Backend Integration & Deployment Pipeline

Implemented Features and Design Decisions

The Flask application was built out with Weather and Station Read APIs returning properly formatted JSON from the shared MySQL database. JWT-based user authentication was integrated with secure password hashing. A Jenkins CI/CD pipeline was configured with a Kubernetes agent, automating Docker builds and EC2 deployments on every push to main. A Git feature-branch workflow was adopted team-wide. Crucially, the Journey Planner backend endpoint was delivered, closing the open scope question from Sprint 1, though it required significant refactoring to handle edge cases and correct route-calculation logic.

Burndown Chart



The burndown chart displays a staircase pattern which directly reflects the project requirement to update the chart every two days. Despite these mandated reporting intervals, the actual remaining effort dropped below the ideal available trend line by Day 4 and remained beneath it for the rest duration of the sprint. This demonstrates that a strong development velocity was maintained by the team. All sprint goals were completed by Day 10.

Sprint Review

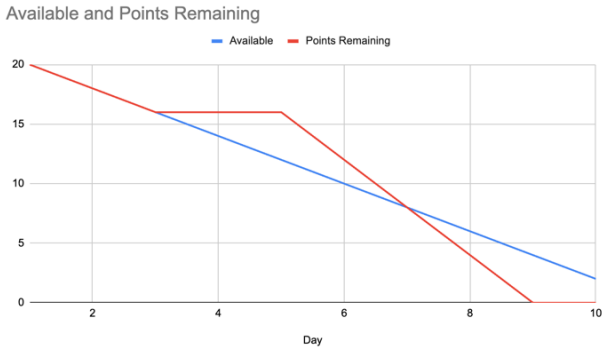
1. **API Demonstration:** Showcased the operational Weather and Station Read APIs successfully returning properly formatted JSON data from both live external APIs and the local MySQL database.
2. **Security & Authentication:** Demonstrated the user authentication flow, highlighting the successful generation of JWT tokens upon login and the secure hashing of passwords in the database.
3. **Pipeline Verification:** Conducted a live demonstration of the CI/CD pipeline. A code push was shown triggering a Jenkins build, which subsequently deployed the updated Flask application to the EC2 instance automatically.
4. **Scope Confirmation:** Confirmed the successful inclusion and basic backend functionality of the Journey Planner endpoint, finalizing the open discussion from Sprint 1.

6.4 Sprint 3 — Additional Features & Frontend Implementation

Implemented Features and Design Decisions

Three AI chat features were implemented using the Langchain framework and the Aliyun Qwen API, with chat history persisted in new database tables and displayed on login. The React frontend was updated to show available bike and stand counts on station hover, and data is now pre-fetched on page load to eliminate display latency. The database was migrated from a local instance to Amazon RDS to overcome EC2 memory constraints and improve query performance. The CI/CD pipeline was verified end-to-end with Docker containerisation confirming a stable, reproducible deployment process.

Burndown Chart



This 10-day burndown chart tracks an initial 20 story points. Despite a complete progress stall between Days 3 and 5 that put them behind the ideal timeline, the team sharply accelerated their work. They ultimately recovered and finished all tasks a day early on Day 9.

Sprint Review

1. **AI Chat Features:** Successfully implemented the three core AI features using the Langchain framework and Aliyun API. Demonstrating chat history storage and LLM memory through new database tables, with chat history displayed upon user login.
2. **Backend Completion:** The Flask backend application is nearing completion, successfully returning the required data, confirmed through Postman testing.
3. **Frontend Data Display:** Updated the frontend to display available bike and stand data upon hovering over station icons. Data requests are now automatically sent on website load to eliminate display latency.
4. **CI/CD & Deployment:** Verified the continuous integration and deployment process using Jenkins. Confirmed the use of Docker and containerization for a professional, normalized deployment process, keeping the production application updated.

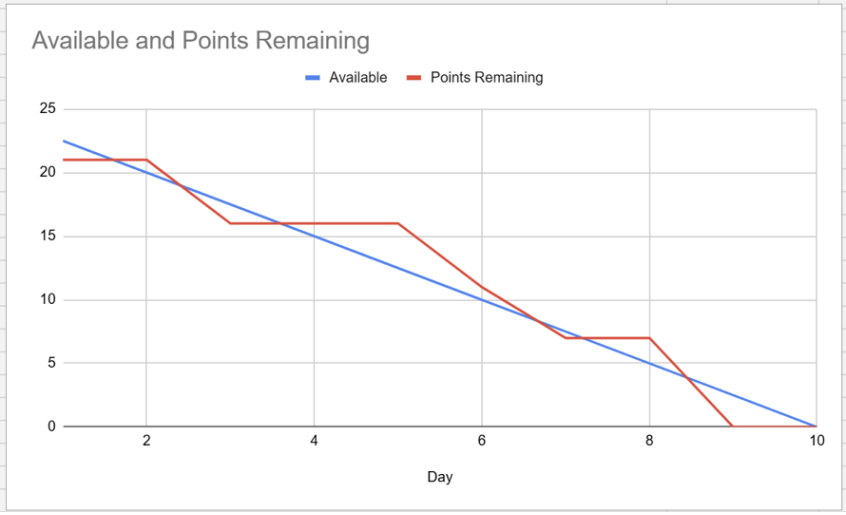
6.5 Sprint 4 — Machine Learning Integration & System Finalisation

Implemented Features and Design Decisions

A Decision Tree model was trained on historical station and weather data, serialised as a .pkl file, and integrated into the Flask backend via a prediction service that preloads the model at startup. The React frontend was updated to fetch and render predictions dynamically, with a brief cold-start latency on the

first request acknowledged and documented. UI styling was refined for consistency across all pages. Backend test coverage reached 97% with pytest, and the CI/CD pipeline was updated to download ML artefacts from Hugging Face Hub during the build stage.

Burndown Chart



The Burndown chart illustrates a steady progression with a few plateaus, reflecting the complexity of our final integration tasks. Our "Points Remaining" line stayed slightly above the "Available" trend line during the middle of the sprint (Days 3-5) as the team was deeply involved in fine-tuning the machine learning algorithms and resolving frontend-backend API formatting issues. However, following a steep drop around Day 7, we regained our high velocity and successfully reached zero remaining tasks on Day 9, one day before the sprint deadline. This efficient completion allowed us to dedicate the final day strictly to code refactoring, ensuring UI consistency, and compiling our final project report.

Sprint Review

1. **Application Demonstration:** Presented the fully functional web application, showcasing the seamless integration between the React frontend and Flask backend. We demonstrated the refined UI styling, highlighting how the consistent design provides a smooth user experience across all pages.
2. **Machine Learning Verification:** Demonstrated the live predictive feature powered by our exported .pkl model. We showed how the frontend fetches data from the backend API and dynamically renders the prediction results. We also explained the slight "cold start" latency during the first click and proved the fast, seamless performance of all subsequent requests.
3. **Quality Assurance & Next Steps:** Highlighted our achievement of 97% unit test coverage, proving the stability and reliability of our codebase. Since all core development is now complete, we discussed our strategy for the final 2-week phase, agreeing to focus strictly on code refactoring, UI/UX beautification, and compiling the final project report.